

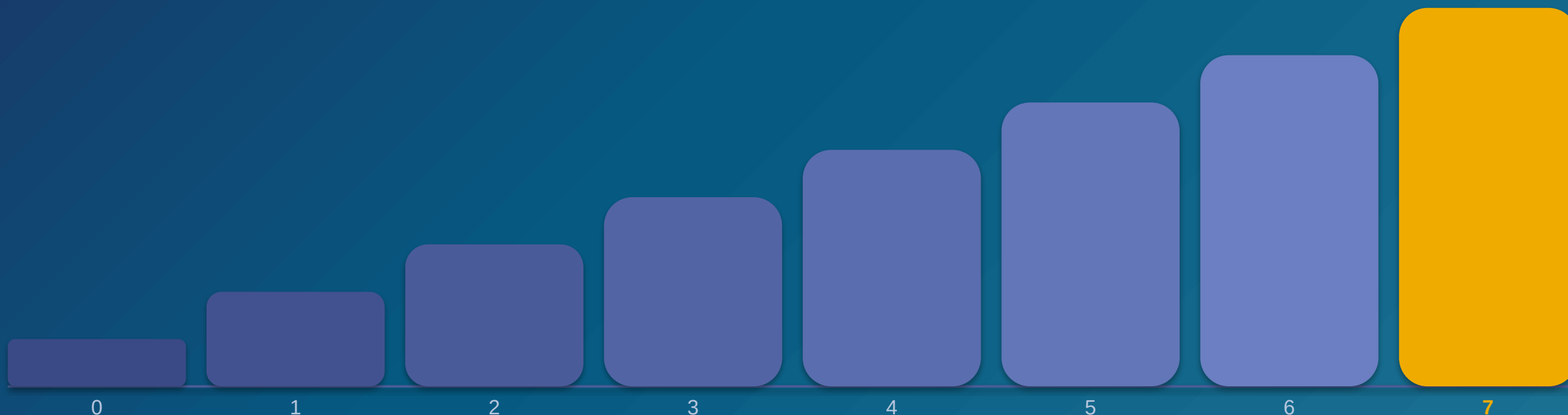
Сборка кодинг-агента

Один сквозной кейс, две развилки харнесса на трёх реальных решениях

04

«У вас репозиторий и агент, который в нём работает — какой блок конфигурации добавить первым, и как понять, что следующий добавлять ещё рано?»

ПУСТАЯ ДИРЕКТОРИЯ → СЕМЬ ДОБАВЛЕНИЙ, ПОКА БЕЗ ИМЁН



Часть требований видно, открыв редактор. Часть — нет, сколько угодно смотри в код

ВИДНО В КОДЕ

НИГДЕ НЕ БУДЕТ ВИДНО

«Заказчик просит лендинг с формой заявки на демо-урок. Одна страница, два поля, кнопка. Назовите вслух: что вообще должно быть у этой задачи, чтобы вы могли сказать ‘готово’ и показать её заказчику? Не как это устроено внутри — а что должно существовать.»

Шесть требований делятся ровно пополам: три увидит в коде любой, три не будут видны нигде

ВИДНО В КОДЕ

поля формы: имя и email, оба обязательные

чем собирается — сборка в статику

чем проверяется — тест: форма не отправляется с пустыми полями

НИГДЕ НЕ БУДЕТ ВИДНО

что считается «готово»

чего агент не делает сам — деплой без явного запроса

как убедиться руками, что заявка реально доставлена

3 + 3

РАЗДЕЛ 0 · СКВОЗНОЙ КЕЙС

Репозиторий буквально пуст, а агент не задал ни одного уточняющего вопроса — и то, чего он не знал, решил за вас молча

```
$ ls -la signup-landing/  
total 8  
drwxr-xr-x 2 harness harness 4096 Sep 20 12:45 .  
drwxr-xr-x 3 harness harness 4096 Sep 20 12:45 ..
```

реальный вывод, не макет

«собери лендинг с формой заявки на демо-урок: статическая HTML-страница, сборка Vite (`npm run build` → `dist/`), поля формы — имя и email, тест на Playwright, который проверяет, что форма не отправляется с пустыми обязательными полями (`tests/form.spec.ts`)»

«Уточняющий вопрос не задавался. Все параметры задачи... были однозначно выводимы из текста задачи, поэтому работа была начата сразу, без уточнений.»

что создал

```
index.html  
src/main.js  
package.json  
tests/form.spec.ts
```

из финального ответа

```
npm install  
npm run build      # → dist/  
npm run test:e2e   # playwright test
```

Первый блок конфигурации выбирают из восьми вариантов на экране — и «ничего» среди них не менее весомый вариант, чем остальные семь

«Перед вами репозиторий, который агент только что собрал, и вам с этим же агентом доделывать этот сайт дальше, неделями. Выберите, какой блок конфигурации вы добавите первым — из карточек ниже — так, чтобы он закрывал реальную боль, а не запас на будущее.»

файл
инструкций

память

хук

скилл

МСП-
доступ

субагент

процесс
проверки

ничего

разбор — на следующем слайде

Бремя доказательства лежит на том, кто усложняет, а не на том, кто остаётся внизу

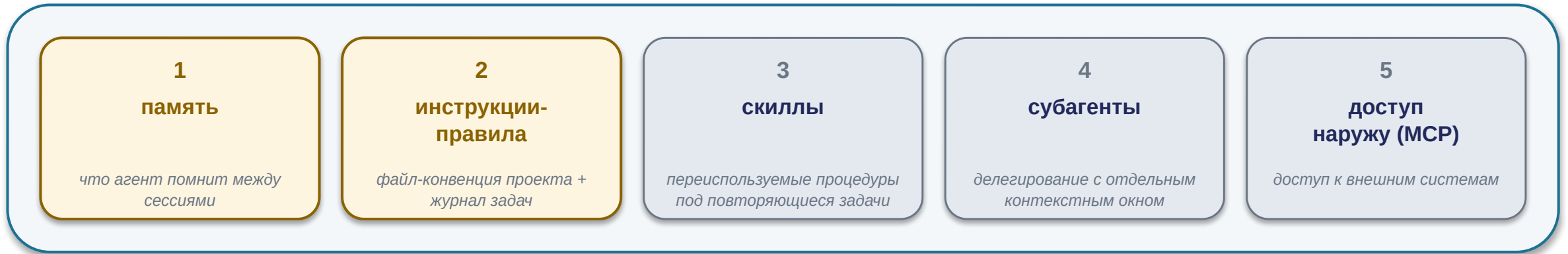
вопрос для контекста: «Какой блок конфигурации добавить первым?»

Прозвучавший вариант	Где это работает	Почему здесь ещё рано — и что вместо
«Файл инструкций — так делают в хороших проектах»	Когда есть накопленное знание, которого нет в коде	«Так делают» — не триггер, а инерция. Вместо: дождаться сигнала. Через пятнадцать минут этот ответ станет верным — но потому, что сработает конкретный сигнал
«Хук — поставлю защиту заранее, на будущее»	Когда риск конкретной команды уже стал реальным	Вы ещё не знаете, какие команды в этом проекте рискованны: защищаете гипотезу, а не факт. Вместо: назвать команду и инцидент, от которого защищаетесь
«MCP — подключу к трекеру задач, пригодится»	Когда доступ наружу нужен повторно и из нескольких сессий	Самый дорогой из семи блоков — по контексту и по поверхности атаки. Вместо: обычная команда в терминале, пока не доказано, что её не хватает
«Ничего»	На нулевой ступени — всегда	Не рано: это и есть ответ. Перестает быть верным ровно тогда, когда появится первое реальное расхождение между тем, что знаете вы, и тем, что знает агент

бремя доказательства лежит на том, кто усложняет, а не на том, кто остаётся внизу

РАЗДЕЛ 0 · ОПОРНАЯ КАРТА

Из пяти слотов экипировки агента сегодня подробно разбираются два — инструкции-правила и память; остальные три и оба практических слоя — дальше, Семинаром 5



Сегодня подробно — два. Дальше, Семинар 5: скиллы, субагенты, доступ наружу + хук, процесс.

«Оставайтесь на самой нижней ступени, которая закрывает требование задачи. Поднимайтесь на следующую только тогда, когда можете назвать требование, которое текущая ступень не закрывает. Каждый подъём оплачивается новой стоимостью, новыми режимами отказа и новой поверхностью атаки.»

Раздел 1 · файл инструкций

- Кейс 1.1 · роль агента и репозиторий
- Кейс 1.2 · базовые процессы: гейты и цикл улучшения
- Кейс 1.3 · вложенные файлы по подпапкам

ОДИН ФАЙЛ, ТРИ РАЗВИЛКИ



Кейс 1.1 · роль агента и репозиторий

Что писать в пустой файл, пока не случилось ни одной реальной сессии

1 КОНТРОЛИРУЕМЫЙ ЭКСПЕРИМЕНТ · 1 ЧЕСТНЫЙ ПРОБЕЛ



КЕЙС 1.1 · РОЛЬ И РЕПОЗИТОРИЙ

Первый инстинкт — открыть пустой файл и начать с описания структуры репозитория; привычка сильная, но это ровно та фраза «так делают в хороших проектах», которую занятие проверяет впервые

дерево на входе в кейс

```
index.html  
package.json  
src/main.js  
tests/form.spec.ts
```

Разработчик открывает пустой CLAUDE.md и по привычке тянется первым делом написать «Repository overview» — из чего собран сайт, где лежит форма, какая структура папок.

«В одном из прошлых проектов раздел «Overview» пережил рефакторинг и два месяца спустя продолжал описывать структуру, которой больше не существовало — агент один раз отредактировал не тот файл, поверив описанию, а не факту.»

Repository overview измеримо не помогает и стоит контекста — кроме единственного подтверждённого исключения: репозитория без другой документации вообще

Источник	Тезис	Сила доказательства
Gloaguen et al., arXiv:2602.11988	«Repository overviews... are not helpful» — «шагов до первого релевантного файла» не падает; cost +20–23% без прироста success	Сильное: контролируемый эксперимент с честным третьим плечом «файла нет вообще»
Та же работа, абляция	Удалили существующую документацию (README) — LLM-сгенерированный overview стал полезен (+2,7%)	Подтверждает механизм: overview вреден потому, что дублирует то, что уже доступно иначе
Lulla et al., arXiv:2601.20404	AGENTS.md с фокусом на недискаверабельные конвенции: −28,6% runtime, −16,6% tokens (124 PR)	Умеренно-сильное «за» файл — но это НЕ про общее описание структуры
Shepard & Albrecht, arXiv:2606.20512	Статичное written-once guidance (28,3% resolve) хуже динамически верифицированного (33,0%, p<0,001)	Косвенное — подтверждает логику «непроверенное статично хуже»

это явление называют *presence paradox* — сам факт присутствия текста в контексте создаёт свою стоимость независимо от того, использует ли его агент (термин используется дальше без повторного объяснения)

«Прямых экспериментов с намеренно устаревшим или неверным repository overview не найдено — это пробел в исследованиях, не установленный факт. Анекдот коллеги — правдоподобный, но не измеренный случай.»

Перед пустым CLAUDE.md выбор из пяти вариантов — и «пока ничего» среди них не менее весомый вариант, чем остальные четыре

«Перед вами пустой CLAUDE.md для signup-landing. Выберите, что из перечисленного вы включите в файл прямо сейчас, до первой реальной сессии агента — из карточек ниже.»

подробное описание
структуры репозитория

список технологий
и зависимостей

инструкцию, где лежит
форма и как её найти

неочевидную
конвенцию,
которую агент не
угадает сам

пока ничего — до
первого реального
сигнала

разбор — на следующем слайде

Правильный ответ на этом шаге — не какой текст написать, а пока никакой: файл остаётся пустым, потому что триггер ещё не сработал

вопрос для контекста: «Что включить в файл прямо сейчас?»

Вариант	Где это работает	Почему здесь ещё рано — и что вместо
«Подробное описание структуры репозитория»	Почти нигде для агента — разве что документация для новых людей в команде	Агент выведет структуру сам за секунды; описание устареет при первом рефакторинге. Эксперимент: +20–23% cost, без прироста success. Вместо: не писать
«Список технологий и зависимостей»	Когда версии критичны и не видны из package.json (редко)	В большинстве случаев тоже выводимо за секунды. Вместо: не писать, если версии видны в манифесте
«Инструкция, где лежит форма и как её найти»	Почти нигде на таком маленьком репозитории	Четыре файла, вся структура на одном экране — находимость не проблема
«Неочевидная конвенция, которую агент не угадает сам»	Когда конвенция реально есть и реально невыводима	На этом шаге такой конвенции ещё нет — проект только начался. Появится — записать её, и только её
«Пока ничего — до первого реального сигнала»	Здесь и сейчас	Целевой ответ. Перестает быть верным, когда появится конкретная невыводимая вещь — не раньше

Всё, что агент выведет из кода за секунды, в файл не идёт никогда — не «пока рано», а вообще. Общее описание роли/архитектуры оправданно только там, где нет вообще другой документации; у signup-landing она будет — значит, оснований нет.

CLAUDE.md остаётся пустым — не из лени, а потому что нет сигнала; первый настоящий сигнал придёт через неделю

дерево репозитория – без изменений

index.html

package.json

src/main.js

tests/form.spec.ts

CLAUDE.md – не создан

«Файл пуст не потому, что мы ленивы, а потому что нет сигнала. Первый настоящий сигнал появится через неделю.»

Кейс 1.2 · базовые процессы

Первая реальная боль — заявка не доходит, тест зелёный

4 ПРЕПРИНТА-КОНСЕНСУС · 1 КОНТРИНТУИТИВНЫЙ РЕЗУЛЬТАТ



КЕЙС 1.2 · БАЗОВЫЕ ПРОЦЕССЫ

Заказчик пишет: заявка не пришла — тест зелёный, «готово» сказано, а тест проверяет только пустые поля; записанный текстовый гейт не изменил поведение при повторе

Блок 1. Заказчик: «отправил заявку — она не пришла». Агент читает код, правит обработчик в `src/main.js`, прогоняет автотест — зелёный — отчитывается: «Готово». `tests/form.spec.ts` проверяет ровно одно: что форма не отправляется с пустыми полями. Про доставку он не знает ничего. Заказчик пишет снова: не доходит.

Блок 2. Команда записывает в `CLAUDE.md` текстовый гейт: «Всегда прогоняй тесты и вручную проверяй результат перед 'готово'». Неделю спустя, новая сессия, другой баг: автотест зелёный — снова «готово», руками не проверено.

«Правило было. Поведение — нет.»

КЕЙС 1.2 · ИССЛЕДОВАНИЕ

Текстовый гейт без тулинга не просто бесполезен, а вреден — регрессии выросли почти до 10%; self-repair с реальной проверкой работает, голая самокритика систематически вредит

исследование — гейты

Источник	Тезис	Сила
SWE-Gate/SpecBench/CapCode/Verification Horizon (4 препринта 2609–2606)	Согласие 4+ независимых 2026-препринтов: агенты систематически «обходят» видимые тестовые гейты, разрыв растёт со сложностью задачи	Сильное — межисследовательский консенсус
TDAD, arXiv:2603.17973	Чисто процедурная инструкция «сначала пиши тесты» без тулинга увеличила регрессии до 9,94% — хуже, чем вообще без инструкции	Сильное и контринтуитивное
Rethinking Agent-Generated Tests, arXiv:2602.07900	Манипуляция промптом ради частоты тестов — без значимого эффекта на resolution rate (6 моделей, SWE-bench Verified)	Умеренное — нулевой результат
Agent Scaffolding Beats Model Upgrades	Архитектура харнесса (test-gated retries, CI-enforcement) даёт +20 п.п.; смена модели — ~1 п.п.	Сильное — механизм против модели

исследование — цикл «улучшение»

Источник	Тезис	Сила
Self-planning, arXiv:2303.06689	Планирование перед кодом: +25,4% pass@1 (HumanEval/MBPP, не repo-scale)	Умеренно-сильное «за»
LLMs Cannot Self-Correct, ICLR'24; CRITIC, 2305.11738	Самокоррекция без внешнего сигнала систематически ухудшает результат	Сильное «против» голого self-review
arXiv:2604.10508	Self-repair с внешним сигналом (реальный прогон теста/компилятора): +4,9...+30 п.п.	Сильное «за» — с реальной проверкой

«Нет контролируемого эксперимента именно на изолированную фразу-гейт в CLAUDE.md — только best-practice наблюдения практиков.»

Гейт-фраза уже есть и не сработала — выбор из пяти вариантов, что делать дальше, чтобы реально сработало в следующей сессии

«У вас гейт-фраза в CLAUDE.md уже есть, а поведение агента не изменилось — заявка снова не проверена руками. Выберите, что вы сделаете дальше — из карточек ниже — так, чтобы это реально сработало в следующей сессии.»

перепишу фразу
капсом / жёстче

продублирую фразу
вторым пунктом

заведу отдельный
файл-чеклист рядом

передам проверку
механизму вне текста

смирюсь — сработает
не всегда, это норма

разбор — на следующем слайде

Честный ответ сегодня: мы всё равно запишем гейт текстом, потому что это лучше, чем ничего, но прямо скажем себе, что он ненадёжен

вопрос для контекста: «Гейт не сработал — что дальше?»

Вариант	Где работает	Почему рано — и что вместо
«Капсом / жёстче»	Нигде системно	TDAD: усиление формулировки без тулинга не помогает и иногда вредит — регрессии выросли до 9,94%. Формулировка — не механизм
«Продублирую вторым пунктом»	Иногда чуть повышает шанс прочитать	Правило остаётся просьбой. Каждая лишняя строка стоит контекста в каждой сессии (presence paradox, кейс 1.1)
«Отдельный файл-чеклист»	Помогает человеку помнить, слабо помогает агенту	Тот же класс решения — снова текст, снова просьба, не обязательна к построчному следованию
«Передам механизму вне текста»	Системно работает: CI/harness-enforcement даёт +20 п.п.	Верное направление — но сам механизм не тема сегодня, это Семинар 5. Сегодня фиксируем: текст сам по себе ненадёжен
«Смирюсь, это нормально»	Приемлемо для некритичных предпочтений	Для правила, чьё нарушение стоит дорого — «смирюсь» означает решение не решать

Правила в промпте — это просьбы, правила в коде — это законы: текстовый гейт — просьба, и сегодня мы записываем именно просьбу. Приемлемый первый шаг, пока нарушение редкое; нарушено дважды на критичном действии — сигнал для эскалации к механическому уровню (Семинар 5). Цикл «улучшение»: агент, смотрящий сам на свой код, — не делать так; агент, прогоняющий реальную проверку, — делать так всегда.

Итоговый файл — тринадцать строк: критерий готово, гейт-предупреждение, честная оговорка предела; AGENTS.md — симлинк на тот же файл

CLAUDE.md

signup-landing: статический лендинг с формой заявки на демо-урок.
Готово = npm run build код 0, npx playwright test зелёный,
форма реально отправляет заявку на проде.

Safety / scope boundaries

- Никогда не запускать деплой на прод без явного запроса.

Build, test, verify

- To verify: открыть dist/index.html после сборки, вручну отправить форму с заполненными и с пустыми полями.

1

Файл грузится в начало каждой сессии целиком (официальный ориентир — до 200 строк, наш итог — 13)

2

@-импорты организуют файл, но не экономят контекст — импортированное всё равно грузится целиком

3

AGENTS.md — симлинк на тот же файл (ln -s CLAUDE.md AGENTS.md). Острый край: cp -R без -P на macOS разыменовывает симлинк во вторую копию молча — копировать через cp -a / git clone / git archive

Кейс 1.3 · вложенные файлы

Месяц спустя репозиторий подрос — дробить файл или нет

1650 СЕССИЙ · 5 ЗАДОКУМЕНТИРОВАННЫХ ISSUE



КЕЙС 1.3 · ВЛОЖЕННЫЕ ФАЙЛЫ

Разработчик выносит правила тестов в tests/CLAUDE.md и ждёт, что агент подхватит его сам, когда работает именно в этой папке — логично звучит, но стоит проверить, что там реально происходит с загрузкой

месяц спустя: tests/ подрос

tests/

form.spec.ts

wizard.spec.ts

page-objects/form.page.ts

своя конвенция именования + Page Object

«Вынесу эти правила в отдельный tests/CLAUDE.md — короче, релевантнее контексту, корневой файл не разбухнет.» Кладёт файл в подпапку, ожидая автоматической подхватки.

«Логично звучит. Давайте проверим, что там на самом деле происходит с загрузкой.»

КЕЙС 1.3 · ИССЛЕДОВАНИЕ

Вложенный файл грузится on-demand, только по факту чтения — не по факту нахождения в папке; контролируемое исследование не нашло пользы от разбиения, а практика показывает регулярные отказы вплоть до архитектурного предела

Механизм (офиц. документация Claude Code): корневой и все родительские CLAUDE.md грузятся при старте сессии автоматически. Вложенные грузятся on-demand — только когда агент реально читает файл оттуда. /context — единственный надёжный способ проверить, что реально загружено.

Источник	Тезис	Сила
McMillan, arXiv:2605.10039 (1650 сессий Claude Code CLI)	Ни одна из 4 структурных переменных не дала значимого эффекта после multiple-testing correction; для size/conflict — свидетельство ОТСУТСТВИЯ эффекта	Сильное и контринтуитивное
Claude Code #2571	Вложенный CLAUDE.md не подхватывается автоматически, воспроизводимо; закрыт «not planned»	Прямое наблюдение
Claude Code #6972	Документация вводит в заблуждение — пользователи ждут eager-загрузку, на деле on-demand	Прямое наблюдение
Cursor, форум	Сотрудник Cursor признал баг в nested-функциональности	Прямое, другой инструмент
Codex CLI #13288, #12115	Ненадёжная подгрузка без явного --cd	Прямое, другой инструмент
GitHub Copilot CLI #3051	Не баг, а документированный структурный предел: discovery только вдоль прямого пути cwd → git-root	Самое жёсткое: архитектурное ограничение

Как гарантировать, что агент реально прочитает конвенции из tests/CLAUDE.md — выбор из пяти вариантов

«Выберите, как вы гарантируете, что агент реально прочитает конвенции из tests/CLAUDE.md, когда работает именно в этой папке — из карточек ниже.»

положу файл в tests/
и доверюсь
автозагрузке

явно попрошу агента
прочитать файл в
начале задачи

продублирую
содержимое
в корневом CLAUDE.md

проверю через /context,
что реально загружено

объединю всё в один
файл в корне, пока не
мешает

разбор — на следующем слайде

КЕЙС 1.3 · РАЗБОР

Редкое сочетание: контролируемое исследование не нашло пользы даже там, где здравый смысл её обещал, а практика показывает регулярные отказы вплоть до архитектурного предела

вопрос для контекста: «Как гарантировать, что файл прочитан?»

Вариант	Где работает	Почему рано — и что вместо
«Доверюсь автозагрузке»	Никогда гарантированно	Вложенные файлы грузятся on-demand, только по факту чтения. Задocumented отказы — системная категория: #2571, #6972, Cursor, Codex CLI, жёсткий предел Copilot CLI #3051
«Явно попрошу прочитать в начале задачи»	Работает, требует ручной дисциплины каждый раз	Надёжнее автозагрузки, но перекладывает ответственность на человека — то, от чего мы уходили в кейсе 1.1
«Продублирую в корневой»	Работает всегда — корень грузится при старте	Снова presence paradox: продублированная строка стоит контекста в каждой сессии, в любой папке
«Проверю через /context»	Единственный надёжный способ узнать факт	Диагностика, не решение — проверяет, сработало ли, не гарантирует в следующий раз
«Объединю в один файл в корне, пока не вырос»	Здесь и сейчас — репозиторий маленький	Целевой ответ. Исследование (1650 сессий) не нашло значимого эффекта ни у одной структурной переменной

Пока корневой файл помещается в разумный объём (до пятидесяти-семидесяти строк), дробление по подпапкам не нужно никогда. Даже при росте объёма дробить стоит только после того, как явно проверено — через /context или наблюдаемое поведение, — что инструмент реально грузит вложенный файл.

КЕЙС 1.3 · РЕШЕНИЕ

Репозиторий остаётся с одним корневым файлом; три кейса — три решения знания, известного заранее, но есть знание другого рода, которое появляется по ходу работы — это раздел второй, память

дерево репозитория — без изменений

AGENTS.md

CLAUDE.md

index.html

package.json

src/main.js

tests/form.spec.ts

tests/CLAUDE.md — не появляется

«Три кейса, один и тот же файл: мы решили, чего в него не писать заранее; написали то, что узнали на реальной боли; и решили не дробить его, пока это не доказанная польза. Всё это — знание, которое у вас уже было до работы, записанное заранее. Но есть знание другого рода: то, что появляется по ходу самой работы. Его нельзя записать заранее, потому что вы его ещё не знаете. Это раздел второй — память.»

Раздел 2 · память

Кейс 2.1 · плоский файл

Между сессиями агент не хранит ни бита — если это не лежит на диске

3 КЕЙСА ПО ВОСХОДЯЩЕЙ СЛОЖНОСТИ



КЕЙС 2.1 · ПЛОСКИЙ ФАЙЛ

Уже отклонённое решение всплывает снова, потому что разговор, в котором его приняли, закончился — а его содержимое нигде не осталось

Третья сессия за две недели. Заказчик прислал заявку с опечаткой в домене почты, форма её приняла. Разработчик просит доработать валидацию. Агент предлагает подключить стороннюю библиотеку валидации форм — ту самую, которую уже обсуждали и отклонили: форма из двух полей, хватает нативных `required/pattern` и двадцати строк своего кода.

Агент предлагает её так, будто вопрос никогда не поднимался. Для него он и не поднимался.

«Агент не нарушил правило. В файле инструкций этого правила не было и не могло быть — решение родилось в разговоре, а разговор кончился. Это не «забыл»: модель не имеет состояния между вызовами.»

КЕЙС 2.1 · ИССЛЕДОВАНИЕ

Presence paradox из кейса 1.1 применяется и здесь: то, что лежит в DECISIONS.md, стоит контекста в каждой сессии независимо от того, нужна ли эта запись сегодня

Кейс 1.1: Gloaguen et al. — сам факт присутствия текста в контексте создаёт стоимость (+20–23%), независимо от того, помогает ли он решить задачу. Мы назвали это presence paradox.

DECISIONS.md — не исключение. Каждая строка грузится в контекст в каждой следующей сессии, не только когда она реально нужна. Тридцать записей о разных темах — цена уже заметна, даже если задача касается только одной из них.

прямого измерения именно на журналах решений не проводилось — это перенос уже установленного механизма на новый объект, не отдельное новое исследование

если запись не бесплатна — точно ли туда идёт вообще всё, что можно туда положить?

Выберите, куда записать отклонённое решение — из карточек ниже — так, чтобы не объяснять его в третий раз

напоминание: отклонённая библиотека валидации всплыла снова — решение родилось в разговоре и нигде не осталось

«Вы второй раз за две недели объясняете агенту, почему в этой форме нет сторонней библиотеки валидации. Выберите, куда записать это решение — из карточек ниже — так, чтобы не объяснять его в третий раз, и обоснуйте, почему не в тот файл инструкций, который мы только что завели.»

**дописать
в CLAUDE.md**

**сказать агенту
«запомни это»**

**завести
DECISIONS.md**

**поставить систему
памяти**

**ничего — сам
запомнит**

Командное решение с обоснованием идёт в DECISIONS.md, личное предпочтение — в авто-память, выводимое из кода — никуда

вопрос для контекста: «Куда записать отклонённое решение — и почему не в файл инструкций?»

Вариант	Где это работает	Почему здесь ещё рано — и что вместо
«Допишу в CLAUDE.md»	Механически сработает: файл читается каждую сессию	Смешивает стабильные правила и растущий лог решений. На десятке решений файл выйдет за ориентир в 200 строк. Вместо: развести слои по авторству
«Ничего, агент сам запомнит»	Нигде: это не настройка, которую забыли включить	Модель не имеет состояния между вызовами по конструкции. Вместо: решить, кто и куда пишет
«Скажу агенту ‘запомни это’»	Для личного предпочтения — идеально	Половина ответа. Уходит в авто-память: машинно-локальный слой без синхронизации и код-ревью
«Заведу DECISIONS.md»	Для командного решения с обоснованием — именно то, что нужно	Вторая половина ответа. Требуется писать «почему», а не «что»
«Поставлю систему памяти»	На десятках-сотнях записей, нескольких пользователях	Рано на порядок: на малых корпусах простой файл регулярно конкурентен со сложными системами

целевой ответ — оба механизма разом, с разделением ролей

Не всё, что узнал агент, вообще стоит записывать — третья корзина такая же законная, как первые две

АВТО-ПАМЯТЬ

Разработчик просит отвечать
коротко, без преамбул

DECISIONS.MD

Не подключаем стороннюю библиотеку
валидации — форма из двух полей

Своего бэкенда не делаем — заявка
уходит на внешний сервис приёма форм

НИКУДА

Обработчик формы лежит
в src/main.js

19 сентября тест упал из-за
таймаута, увеличили ожидание

собранная запись — реальный файл демо-репозитория

2026-09-20 — не подключаем библиотеку валидации форм

Форма — два поля (имя, email), хватает нативных required/pattern в
index.html. Сторонняя библиотека — лишняя зависимость ради этого объёма.

Два параллельных механизма, не один: авто-память живёт вне репозитория, DECISIONS.md — git-трекнутый лог «почему» внутри него

DECISIONS.md — целиком

Decisions

Append-only log of why, in date order.

2026-09-20 — не подключаем библиотеку валидации форм

Форма — два поля (имя, email), хватает нативных required/pattern в index.html. Сторонняя библиотека — лишняя зависимость и лишние килобайты в сборке ради этого объёма.

дерево кейса 2.1

AGENTS.md

CLAUDE.md

DECISIONS.md

index.html

package.json

src/main.js

tests/form.spec.ts

diff создания файла: +DECISIONS.md (новый), git add + commit — команда, а не агент, решает, когда коммитить журнал решений.

Разница между слоями не в формате — оба markdown, — а в том, кто инициирует запись и кто её потом увидит

	Файл инструкций	Авто-память	DECISIONS.md
Кто автор	вы, заранее	агент, по ходу	вы, в момент решения
Где живёт	в репозитории, git	вне репозитория, на этой машине	в репозитории, git
Что грузится	целиком, каждую сессию	индекс: первые 200 строк / 25 КБ	ничего автоматически
Проходит код-ревью	да	нет	да
Переживает смену машины	да	нет	да
Что туда идёт	стабильные правила и границы	предпочтения, подтверждённые подходы	решения и их «почему»

индекс памяти — только индекс · агент сам пропускает выводимое из кода · сжатие контекста переживают только файлы на диске

Память по умолчанию читается как доверенный контекст, а не как данные — и включение памяти не обязательно улучшает результат



Отравление памяти — SpAIware (сентябрь 2024)

Внешний текст с вложенной инструкцией → долговременная память → активация в последующих сессиях, эксфильтрация переписки. Вектор закрыт в версии 1.2024.247; структурный принцип остался.

≈48% всех последующих обращений к памяти захвачено (MemoryGraft, 2025; до отравления — 0%). В смежном эксперименте 5 подложенных документов управляют выводом в >90% случаев.

Замер	Значение	База сравнения
Популярная система памяти на бенчмарке долговременной памяти	46,0% с памятью против 57,6% без	та же система, память выключена
Шесть систем памяти против «весь текст сессии в один файл»	проигрыш плоскому файлу на 25–67 пунктов	тот же плоский файл
Дискреционность записи	17% (4 задачи из 24) — модель отказалась записать нужный факт	те же 24 задачи

Плоского файла достаточно, пока история решений влезает в контекст целиком и связей между записями ещё нет

- Задача одноразовая, второй сессии по этому контексту не будет.
- Весь нужный контекст помещается в одно контекстное окно без потерь.
- Правила проекта стабильны и укладываются в файл инструкций короче 200 строк.
- Никто не повторяет вручную одну и ту же правку из сессии в сессию.
- Нет нескольких участников, которым нужно делиться контекстом асинхронно.
- Факт выводим из кода или истории изменений.

Плоский DECISIONS.md перестаёт справляться, когда записей набралось настолько много, что найти нужную линейным чтением дольше, чем переобъяснить решение заново, — и между записями появились связи, для которых в append-only логе физически некуда положить.

Кейс 2.2 · структурированная вики-память

Тот же DECISIONS.md, но разросшийся — плоский лог больше не хранит связей

1 КОНТРПРИМЕР СТРУКТУРЕ · 4 ЧЕСТНЫХ ИЗДЕРЖКИ



Файл технически на месте, но пользоваться им дольше, чем не пользоваться — линейный лог не хранит связей между записями

Прошло четыре месяца. Новый разработчик в команде просит агента добавить капчу на форму: появился спам. Агент листает DECISIONS.md целиком (тридцать восемь записей) и предлагает решение, которое противоречит более раннему решению №14: «не добавляем сторонние виджеты в форму, см. запись №3». Агент нашёл запись №3, но не нашёл запись №31 — она отменяет часть решения №3 конкретно для случая антиспама, дописана двумя месяцами позже, в другом конце файла, без единой отсылки назад.

Разработчик открывает файл сам и тратит на линейное чтение тридцати восьми записей больше времени, чем ушло бы на то, чтобы заново передумать вопрос с нуля.

Пример иллюстративный — реального разросшегося DECISIONS.md на 38 записей не захвачено.

Тридцать восемь записей и потерянная связь между двумя из них — повод выбрать, как реорганизовать журнал решений, а не заводить его заново

«DECISIONS.md разросся до тридцати восьми записей, и в нём уже есть решение, которое отменяет часть более раннего решения, — без единой ссылки между ними. Выберите, как реорганизовать этот файл — из карточек ниже — так, чтобы агент быстро находил нужное решение и видел связи между решениями. Обоснуйте выбор.»

оставить как есть —
просто грепать

разбить на
тематические
файлы со ссылками

поставить графовую/
векторную систему
памяти

завести один общий
SUMMARY.md,
переписываемый
целиком

агент сам найдёт
через поиск по
репозиторию

разбор — на следующем слайде

Разбить на тематические файлы со сквозными ссылками — целевой ответ, но не бесплатный: структура сама создаёт новую нагрузку

вопрос для контекста: «Как реорганизовать разросшийся журнал решений?»

Вариант	Где это работает	Почему здесь ещё рано — и что вместо
«Оставить как есть, просто грепать»	Пока записей десятки и они укладываются в контекст целиком	Деградация плоской памяти со временем задокументирована кросс-источниками (~72 ч, -14 п.п.); наш файл уже перерос эту границу
«Разбить на тематические файлы со сквозными ссылками»	Именно этот случай	Целевой ответ, но не бесплатный — структура сама создаёт новую нагрузку (staleness, дублирование, поддержка)
«Поставлю графовую/векторную систему памяти»	На больших многопользовательских корпусах с семантическим поиском	Контрпример прямо про наш масштаб: файловое хранилище обошло graph-based систему на LoCoMo. Рано на порядок
«Один общий SUMMARY.md, переписываемый целиком»	Для совсем маленького проекта	Переписывание целиком убивает append-only гарантию — история решения невосстановима
«Агент сам найдёт через поиск по репозиторию»	Для фактов, выводимых из кода	Решения — не факты кода. «Сам найдёт» — то же заблуждение, что «сам запомнит»

целевой ответ — разбить на тематические файлы со сквозными ссылками, со списком реальных издержек, а не как безусловный апгрейд

Запись явно ссылается на связанную запись — то, чего в плоском append-only логе не было в принципе

`memory/decisions/`

<code>index.md</code>	← оглавление тем + ссылки
<code>validation.md</code>	← решения по валидации формы
<code>backend.md</code>	← решения о том, чего мы НЕ делаем
<code>deploy.md</code>	

`validation.md` — пример записи

2026-09-20 — не подключаем библиотеку валидации форм

Форма — два поля, хватает нативных `required/pattern`.

См. также: 2026-11-30 (антиспам) — это решение НЕ

распространяется на защиту от спама, другой механизм.

2026-11-30 — антиспам через `honeypot`-поле, без капчи

Отменяет часть решения от 2026-09-20 для случая антиспама.

иллюстративный пример, не захваченный вживую — ждёт реальной сессии

КЕЙС 2.2 · ПРОВАЛ И ИЗДЕРЖКИ

Структура — не гарантированный выигрыш и не бесплатный апгрейд: простое файловое хранилище обошло графовую систему памяти, а сама структура добавляет свою собственную нагрузку

почти все источники ниже — свежие непроверенные препринты 2026 года; цифры — «заявлено в препринте», а не установленный факт

Система	LoCoMo (долговременная память)
Простое файловое хранилище (grep/чтение файлов)	74,0%
Mem0 (graph-based)	68,5%

объяснение авторов: модели тренированы на файловых операциях как на самом привычном инструменте

ЧЕСТНО НАЗВАННЫЕ НЕДОСТАТКИ ВИКИ-ПАМЯТИ

Staleness — сводка в оглавлении расходится с содержимым тематического файла. · Duplication — устаревшие версии одного и того же факта накапливаются вместо замены. · Отсутствие каскадного удаления — удалили факт в одном файле, производные записи в других остались жить как верные. · Maintenance burden — кто-то должен реально ходить и поддерживать структуру актуальной.

КЕЙС 2.2 · КРИТЕРИЙ И МОСТ

Единиц-десятков тематических файлов, которые правят один-два человека, достаточно — полноценная система памяти требует замеров на своём масштабе, не общего совета

- Число тематических файлов — единицы-десятки, не сотни.
- Поиск нужной записи — по имени темы или простому grep, а не по смысловой близости.
- Участников, одновременно правящих память, — один-два.
- Никто не просил «найди решения, похожие по смыслу на...».

Смена оси: всё в этом и предыдущем кейсе — про решения, которые живут навсегда. Что происходит с памятью одной задачи, пока она ещё не закончена?

Кейс 2.3 · операционная память

Смена оси: не решения навсегда, а ход одной задачи между её сессиями

21 120 ТРАЕКТОРИЙ · 1 КЕЙС-СТАДИ N=1



КЕЙС 2.3 · ЗАДАЧНАЯ ПАМЯТЬ

Вторая сессия многошаговой задачи начинается не с продолжения, а с повторного анализа с нуля — и с конфликта с уже принятыми по ходу мелкими решениями

Разработчик ставит агенту задачу на несколько сессий: разбить форму signup-landing на пошаговый визард (имя → email → подтверждение), сохранив всю текущую валидацию. Шесть шагов, первая сессия обрывается на середине, контекст сжимается. Вторая сессия: агент, не имея под рукой ничего, кроме кода и сжатой сводки, заново анализирует форму с нуля и предлагает план, который на треть повторяет уже сделанное и на треть противоречит мелким решениям, принятым по ходу первой сессии — например, переписать email-валидацию, которую договорились не трогать.

Разработчик тратит начало второй сессии не на продолжение работы, а на то, чтобы заново объяснить, что уже сделано.

Агент после сжатия контекста или в новой сессии должен продолжить с того места, где остановился, а не начать заново

«Вторая сессия по одной и той же многошаговой задаче начинается с того, что агент заново анализирует форму с нуля — и предлагает переделать то, что уже решили не трогать. Выберите, куда и как записывать ход этой задачи — из карточек ниже — так, чтобы агент после сжатия контекста или в новой сессии продолжил с того места, где остановился. Обоснуйте выбор.»

надеяться на
контекстное окно

записать план
в DECISIONS.md

завести один файл
на задачу

завести папку на задачу
с файлом под каждый
шаг

попросить агента
«продолжи с того
места»

разбор — на следующем слайде

Один файл на задачу — план, лог хода, результаты — целевой ответ для линейной задачи с одним агентом и одним разработчиком

вопрос для контекста: «Куда записывать ход многошаговой задачи?»

Вариант	Где это работает	Почему здесь ещё рано — и что вместо
«Не сжимать, не закрывать сессию»	Для задачи, целиком укладываемой в одну короткую сессию	Контекстное окно конечно; на многошаговой задаче рано или поздно случится сжатие или разрыв сессии — это отсутствие стратегии
«Записать план в DECISIONS.md»	Никогда — не тот слой памяти	DECISIONS.md — про «почему» для решений, которые остаются в силе навсегда; статус одной задачи туда не относится
«Один файл на задачу: план + лог хода + результаты»	Ровно наш случай	Целевой ответ для этого масштаба
«Папка на задачу с файлом под каждый шаг»	Параллельные подзадачи, несколько субагентов, разросшийся лог	Оверинжиниринг для одной линейной задачи здесь и сейчас. Паттерн реально масштабируется до папки
«Попросить ‘продолжи с того места’»	Внутри одной непрерывной сессии, до сжатия	После сжатия или в новой сессии буквально нечего продолжать — просьба адресована памяти, которой не существует

целевой ответ — один файл на задачу, с явным уточнением, что это упрощённая версия паттерна, масштабируемого до папки

КЕЙС 2.3 · РЕШЕНИЕ

Файл задачи ссылается на журнал решений, а не дублирует его — и это осознанное упрощение паттерна, полностью раскрываемого в другом месте

```
.tasks/active/signup-wizard.md
```

```
# Задача: разбить форму на пошаговый визард
```

```
## План
```

1. [x] Разметка трёх шагов (имя / email / подтверждение)
2. [x] Перенос текущей валидации без изменений
3. [] Состояние текущего шага (JS)
4. [] Кнопки «назад»/«далее»
5. [] Тесты на переключение шагов
6. [] Ручная проверка полного прохождения

```
## Лог хода
```

- 2026-09-20: шаги 1–2 закрыты. Договорились НЕ трогать email-валидацию — обсуждалась в DECISIONS.md (2026-09-20), тут перенос.
- 2026-09-22: начат шаг 3, не закончен — контекст сжат в середине.

```
## Результаты (заполняется по завершении)
```

```
—
```

иллюстративный пример, не захваченный вживую — ждёт реальной многошаговой сессии владельца · упрощение до файла — масштабируется до папки, подробнее за пределами этого занятия

Плохой план хуже отсутствия плана вообще — и это подтверждено количественно, а не только интуицией

Источник	Результат
Evaluating Plan Compliance, 21 120 траекторий, SWE-bench Verified/Pro	«bad plans hurt performance more than no plan at all»; комплаенс к плану падает ~на 13 п.п. на сложном бенчмарке
Fresh Memory, Stale Plans, 30 контролируемых живых сценариев	без проверки актуальности плана исполнитель действовал по устаревшему плану во всех 30 случаях (100%)



Кейс-стади «Index Sickness» (n=1, честная оговорка)

Реальный проект, 391 сессия за месяц. Решение — физическое разделение baseline-инструкций и append-only лога сессии. Результат: объём AI-инструкций сократился ~на 75%, рецидивов не было за следующие ~150 сессий. n=1 action research одного автора, не контролируемый эксперимент.

практика «файл/папка на задачу» пока не мейнстрим — 2853 реальных репозитория, доминируют статические context-файлы

КЕЙС 2.3 · КРИТЕРИЙ

Одного файла на задачу достаточно, пока задача линейная и её ведёт один агент и один разработчик — иначе оправдан переход к папке

- Задача линейная: шаги идут по порядку, не параллельно.
- Работает один агент (без субагентов/параллельных веток) и один разработчик.
- Лог хода помещается в разумный файл (сотни строк, не тысячи) без потери читаемости.
- Результатов и артефактов на выходе немного — не нужен отдельный файл на каждый.

Если хотя бы половина пунктов неверна — переход к папке на задачу оправдан.

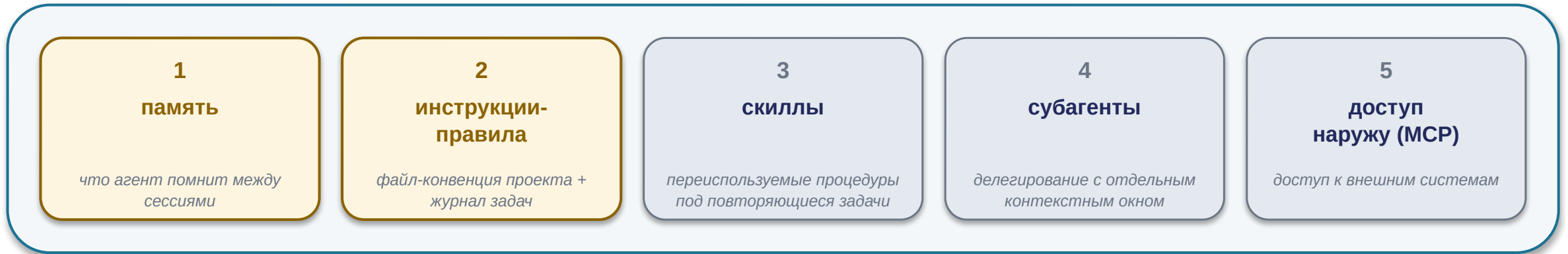
честная оговорка: паттерн «файл/папка на задачу» пока не мейнстрим (2853 реальных репозитория, доминируют статические context-файлы) — практику продвигают отдельные лидеры, это не общепринятая норма

Шесть развилок сегодняшнего занятия — три про файл инструкций, три про память — отвечали на один и тот же скрытый вопрос

Кейс	Развилка	Что мы записали
1.1	Роль агента и общее описание репозитория	Ничего — до первого реального сигнала
1.2	Базовые процессы: гейт «готово» + цикл улучшения	Гейт-фраза текстом в CLAUDE.md
1.3	Вложенные файлы инструкций по подпапкам	Не дробим — один файл в корне
2.1	Плоский файл памяти (DECISIONS.md)	Решение записано в файл + авто-память
2.2	Структурированная вики-память	Тематические файлы со сквозными ссылками
2.3	Операционная задачная память	Один файл на задачу: план + лог + результаты

Всё в правом столбце — текст, который агент читает и может учесть.

Сегодня закрыты подробно два слота харнесса из пяти; оставшиеся три плюс оба механических слоя, которые превращают просьбу в барьер, — материал Семинара 5



Семинар 5: скиллы · субагенты · доступ наружу (MCP) + хук · процесс

Перенесите шесть сегодняшних решений на свой репозиторий — и проверьте, какое из них держится только на просьбе текстом

1.1 → 1.2 → 1.3 → 2.1 → 2.2 → 2.3

роль и репозиторий · базовые процессы · вложенные файлы · плоский файл · вики-память · задачная память

«По каждой развилке — то, что у вас уже принято, это просьба текстом, которую агент технически может обойти? Или нужен барьер?»

«Не вслух. Не на бумаге. Себе.»